

Scientific Computing

L06 Network Algorithms

Thorsten Koch

Applied Algorithmic Intelligence Methods department at ZIB
Chair of *Software and Algorithms for Discrete Optimization*
at TU Berlin



Thomas Eugene Kurtz (February 22, 1928 – November 12, 2024) was an American computer scientist and educator. A Dartmouth professor of mathematics, he and colleague John G. Kemeny set in motion the then **revolutionary concept of making computers as freely available to college students as library books were**, by implementing the concept of time-sharing at Dartmouth College. In his mission to allow non-expert users to interact with the computer, he **co-developed the BASIC programming language and the Dartmouth Time-Sharing System during 1963 to 1964.**

<https://www.knox.edu/news/in-memoriam-thomas-eugene-kurtz-50>



The original version supported the commands LET, PRINT, END, FOR...NEXT, GOTO, GOSUB...RETURN, IF...THEN, DEF, READ, DATA, DIM, and REM. It included basic math instructions, +, -, * and /, as well as the up-arrow for exponents. Exponents took the absolute value of the number before calculation, so to calculate $-X^3$, one had to use $X*X*X$. There was a further problem in the exponent function that treated $-X^2$ as $(-X)^2$ as opposed to the correct order of operations $-(X^2)$. Variable names were limited to a single letter or a letter followed by a digit. All operations were done in floating point.

Additionally, **due to the GE-235's word size being 20-bits and using a six-bit character code, the language enshrined the use of three-letter function names because that allowed the storage of three six-bit characters in a 20-bit word** (using 18 bits). This is why BASIC functions are three letters, like INT or SQR, something that remained in the many varieties of the language long after they left the GE-235.

```
#define O(b,f,u,s,c,a)b(){int o=f();switch(*p++){X u:_ o s b();X c:_ o a
b();default:p--;_ o;}}
#define t(e,d,_ ,C)X e:f=fopen(B+d,_ );C;fclose(f)
#define U(y,z)while(p=Q(s,y))*p++=z,*p=' '
#define N for(i=0;i<11*R;i++)m[i]&&
#define I "%d %s\n",i,m[i]
#define X ;break;case
#define _ return
#define R 999
typedef char*A;int*C,E[R],L[R],M[R],P[R],l,i,j;char B[R],F[2];A m[12*R],malloc
(),p,q,x,y,z,s,d,f,fopen();A Q(s,o)A s,o;for(x=s;*x;x++){for(y=x,z=0;*z&&*y==
*z;y++)z++;if(z>o&&!*z)_ x;}_ 0;}main(){m[11*R]="E";while(puts("0k"),gets(B)
)switch(*B){X'R':C=E;l=1;for(i=0;i<R;P[i++]=0);while(l){while(!(s=m[l]))l++;if
(!Q(s,"\\\"))){U("<","<");U("<=", '$');U(">=", '!');}d=B;while(*F=*s){*s=' ';&j
++;if(j&1||!Q(" \\t",F))*d++=*s;s++;}*d--=j=0;if(B[1]!='=')switch(*B){X'E':l=-1
X'R':B[2]!='M'&&(l=*--C)X'I':B[1]!='N'?gets(p=B),P[*d]=S():(*q=Q(B,"TH"))=0,p
=B+2,S()&&(p=q+4,l=S()-1)X'P':B[5]=''?*d=0,puts(B+6):(p=B+5,printf("%d\n",S
()))X'G':p=B+4,B[2]='S'&&(*C++=l,p++),l=S()-1X'F':*(q=Q(B,"TO"))=0;p=B+5;P[i
=B[3]]=S();p=q+2;M[i]=S();L[i]=lX'N':++P[*d]<=M[*d]&&(l=L[*d]);}else p=B+2,P[
*B]=S();l++;}X'L':N printf(I)X'N':N free(m[i]),m[i]=0X'B':_ 0t('S',5,"w",N
fprintf(f,I))t('O',4,"r",while(fgets(B,R,f))(*Q(B,"\\n")=0,G()))X 0:default:G(
);}_ 0;}G(){l=atoi(B);m[l]&&free(m[l]);(p=Q(B," "))strcpy(m[l]=malloc(strlen(p
)),p+1):(m[l]=0,0);}O(S,J,'=',==,'#',!=)O(J,K,'<','<','>','>')O(K,V,'$',<=,'!',>=)
O(V,W,'+',+,'-',-)O(W,Y,'*',*, '/',/)Y(){int o;_ *p='-'?p++,-(Y):*p>='0'&&*p<=
'9'?strtol(p,&p,0):*p=='('?p++,o=S()),p++,o:P[*p++];}
```

```
10 REM Lunar Lander
20 REM By Diomidis Spinellis
30 PRINT "You aboard the Lunar Lander about to leave
the spacecraft."
40 TO LOAD THIS PROGRAM TYPE "OLD LUNAR.BAS"
60 GOSUB 4000
70 GOSUB 1000
80 GOSUB 2000
90 GOSUB 3000
100 H = H - V
110 V = ((V + G) * 10 - U * 2) / 10
120 F = F - U
130 IF H > 0 THEN 80
135 H = 0
140 GOSUB 2000
150 IF V > 5 THEN 200
160 PRINT "Congratulations! This was a very good
landing."
170 GOSUB 5000
180 GOTO 10
200 PRINT "You have crashed."
210 GOTO 170
1000 REM Initialise
1010 V = 70
1020 F = 500
1030 H = 1000
1040 G = 2
1050 RETURN
2000 REM Print values
2010 PRINT "      Meter readings"
2015 PRINT "      -----"
2020 PRINT "Fuel (gal):"
2030 PRINT F
2040 GOSUB 2100 + 100 * (H > 0)
2050 PRINT V
2060 PRINT "Height (m):"
2070 PRINT H
```

```
2080 RETURN
2100 PRINT "Landing velocity (m/sec):"
2110 RETURN
2200 PRINT "Velocity (m/sec):"
2210 RETURN
3000 REM User input
3005 IF F = 0 THEN 3070
3010 PRINT "How much fuel will you use?"
3020 INPUT U
3025 IF U < 0 THEN 3090
3030 IF U <= F THEN 3060
3040 PRINT "Sorry, you have not got that much fuel!"
3050 GOTO 3010
3060 RETURN
3070 U = 0
3080 RETURN
3090 PRINT "No cheating please! Fuel must be >= 0."
3100 GOTO 3010
4000 REM Detachment
4005 PRINT "Ready for detachment"
4007 PRINT "-- COUNTDOWN --"
4010 FOR I = 1 TO 11
4020 PRINT 11 - I
4025 GOSUB 4500
4030 NEXT I
4035 PRINT "You have left the spacecraft."
4037 PRINT "Try to land with velocity less than 5 m/sec."
4040 RETURN
4500 REM Delay
4510 FOR J = 1 TO 500
4520 NEXT J
4530 RETURN
5000 PRINT "Do you want to play again? (0 = no, 1 = yes)"
5010 INPUT Y
5020 IF Y = 0 THEN 5040
5030 RETURN
5040 PRINT "Have a nice day."
```

Claimed to be a BASIC interpreter that can run the program on the right.

But is very traditional (antique) C, which cannot be compiled by a modern compiler anymore.

Programming:

Convert the code on left of the previous page into some readable Rust that is able to run the program on the right.

Writing:

Write up an explanation of what is to do in the PACE Challenge 2026

<https://pacechallenge.org/2026/maf/>

And make a plan (not actually doing something) how to approach it.

```
fn f() -> bool {  
    print!("f");  
    return true;  
}  
  
fn g() -> bool {  
    print!("g");  
    return true;  
}  
  
fn main() {  
    if f() || g() {  
        print!("x");  
    }  
    if f() && g() {  
        print!("y");  
    }  
    if !f() && !g() {  
        print!("z");  
    }  
    println!(".");  
}
```

What does this program print?

fxfgyf.

This is called “shortcut evaluation”.

- ▶ **Efficiency:** It avoids unnecessary evaluations, improving performance.
- ▶ **Avoiding Side Effects:** If the second condition involves a function call with side effects, those effects won't happen if the first condition determines the result.

Remember: Side-Effects!

A greedy algorithm is any algorithm that follows the strategy of making the locally optimal choice at each stage. In many problems, a greedy strategy does not produce an optimal solution.

For example, a greedy strategy for the travelling salesman problem is the following heuristic: “At each step of the journey, visit the nearest unvisited city.”

This heuristic is unlikely to find the best solution, but it terminates in a reasonable number of steps.

In mathematical optimization, greedy algorithms optimally solve combinatorial problems having the properties of matroids and give constant-factor approximations to optimization problems with the submodular structure.

Capacity: 100

Sort by **profit**

Profit Weight

100 100

99 1

99 1

...

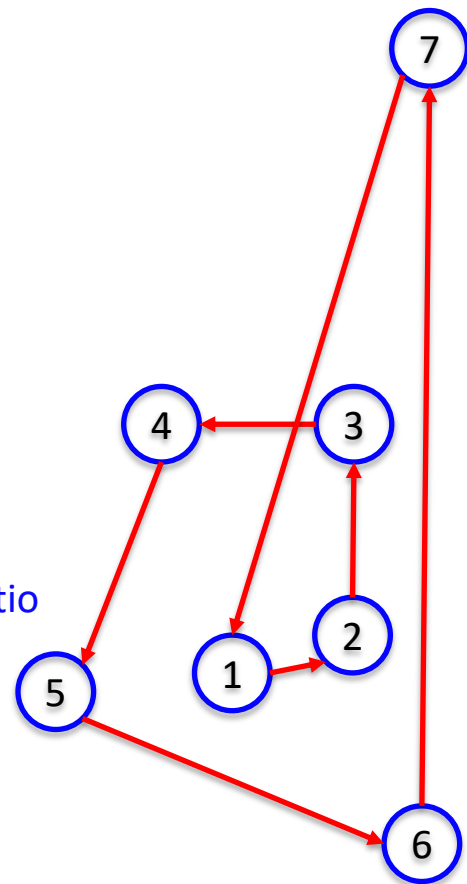
Capacity: 100

Sort by **profit/weight ratio**

Profit Weight

1 1

99 100



Topics 1: Shortest-Path, Spanning-Tree, Min-Cut/Max-Flow, Min-Cost-Flow

1. Hougardy, Vygen: Chapter 9,10
2. Roughgarden: Chapter 7-10 (Part 2)
3. Fischetti, Chapter 7
4. Ahuja, Magnanti, Orlin, Network Flows, Chapter 4-6, 9
5. Williamson, Network Flow Algorithms, Cambridge University Press, 2019, Chapter 1-5
6. Ford, Fulkerson, Flows in Networks, Princeton, 1962
7. Bertsekas, Linear Network Optimization, MIT Press, 1991, Chapter 1
8. Each of the keywords above has a Wikipedia entry and there are many videos about these topics.

Topics 2: Collections

- Read [TRPL Chapter 8](#) Common Collections

- ▷ Networks are called *Graphs* in Mathematics
- ▷ Graphs consist of *Nodes* (or *Vertices*) and the links between the nodes are called (directed) *Arcs* or (undirected) *Edges*
- ▷ Nodes have a *Degree* $\delta(n)$ and *Neighbors*
- ▷ Arcs have a *Head* and a *Tail*, and can have a *Cost* or *Weight*, and a *Capacity*
- ▷ Graphs can be
 - ▶ *Simple, Complete, Bipartite, Layered, Planar, Directed or Undirected, Acyclic, Strongly connected*
 - ▶ have one or more *Connected components*
- ▷ Graphs may contain
 - ▶ *Forests, Trees, (directed) Cycles*, and (directed) *Paths, Eulerian paths*
- ▷ Trees have some special named nodes:
 - ▶ a *root*, and *leaves*, *parents*, and *siblings*.

Student Presentation

We conducted an investigation of the proposed method on the maximum clique problem, which finds the largest complete subgraph of a graph. [...]

To study the scaling of the proposed approach, we considered random max-clique problems with 10, 30, . . . , 130 vertices. For each graph size, we generated 25 random Erdős-Rényi instances with edge densities $p \in \{0.25, 0.5, 0.75\}$ [...]

Let us see what does the above mean?

A random (Erdős-Rényi) graph $G(n, p)$ is obtained constructing a graph with $|V| = n$ vertices and adding each edge $(u, v) \in V \times V, u \neq v$, with probability $p \in [0, 1]$.

https://en.wikipedia.org/wiki/Random_graph

https://en.wikipedia.org/wiki/Erdős-Rényi_model

Can you give me code to generate Erdős–Rényi model graphs in Rust?

💡 Certainly! To generate an Erdős–Rényi graph (either $G_{n,p}$ or $G_{n,m}$ model), you can use the `'rand'` crate to help with the randomization. Below, I'll provide a simple example of generating a $G_{n,p}$ graph where n is the number of nodes and p is the probability of creating an edge between any two nodes.

First, add the `'rand'` crate to your `'Cargo.toml'`:

```
toml
[dependencies]
rand = "0.8.4"
```

Now, here's an example of generating an Erdős–Rényi graph as an adjacency list, where each node has a vector of adjacent nodes:

```
rust
use rand::Rng;
use std::collections::HashSet;

fn generate_erdos_renyi_graph(n: usize, p: f64) -> Vec<HashSet<usize>> {
    let mut rng = rand::thread_rng();
    let mut graph = vec![HashSet::new(); n];

    for i in 0..n {
        for j in (i+1)..n {
            if rng.gen::() < p {
                // Add edge both ways for undirected graph
                graph[i].insert(j);
                graph[j].insert(i);
            }
        }
    }
    graph
}

fn main() {
    let n = 5; // Number of nodes
    let p = 0.5; // Probability of edge creation

    let graph = generate_erdos_renyi_graph(n, p);

    // Print the adjacency list representation of the graph
    for (i, neighbors) in graph.iter().enumerate() {
        println!("Node {}: {:?}", i, neighbors);
    }
}
```

This code will create a simple, undirected Erdős–Rényi graph. The `'generate_erdos_renyi_graph'` function initializes an adjacency list with no edges and then iterates through each possible pair of nodes. For each pair, it generates a random number and compares it with the probability p . If the random number is less than p , it creates an edge by adding each node to the other's adjacency set.

Note that the adjacency list is represented by a `'Vec<HashSet<usize>>'`, which ensures that there are no duplicate edges and that the edges are unordered. Each `'HashSet<usize>'` contains the indices of the nodes that are adjacent to the node with the corresponding index.

The `'main'` function demonstrates how to call `'generate_erdos_renyi_graph'` and prints the resulting graph. Adjust the `'n'` and `'p'` values to generate different graphs.

```
// cargo add rand
```

```
use rand::distributions::{Distribution, Uniform};
```

```
/// Generate random Erdős–Rényi graph
```

```
pub fn erdős_renyi(node_count : usize, probability : f64) -> Graph {
```

```
    let mut rng = rand::thread_rng();
```

```
    let distribution = Uniform::from(0.0 .. 1.0);
```

```
    let mut graph = Graph::new();
```

```
graph.nodes.resize_with(node_count, || { Node::new() });
```

```
for u in 0..node_count {
```

```
    for v in u + 1 .. node_count {
```

```
        if probability > distribution.sample(&mut rng) {
```

```
            graph.add_edge(u as NodeNo, v as NodeNo, 1u32);
```

```
        }
```

```
    }
```

```
}
```

```
graph
```

```
}
```

Possible, but not that great an idea to have non-ASCII chars in names

How would you check that the generated graph has indeed the requested properties?

Given a simple Graph $G = (V, E), E \subseteq V^2$.

A clique is a set of vertices $C \subseteq V$ with $u, v \in C \Rightarrow (u, v) \in E$

The clique number is

$$\max_{C \subseteq V} |C| \text{ with } u, v \in C \Rightarrow (u, v) \in E$$

A set $S \subseteq V$ with $u, v \in S \Rightarrow (u, v) \notin E$ is called independent.

The independence number is

$$\max_{S \subseteq V} |S| \text{ with } u, v \in S \Rightarrow (u, v) \notin E$$

Let K consist of all 2-element subsets of V .

Then $H = (V, K \setminus E)$ is the complement of G .

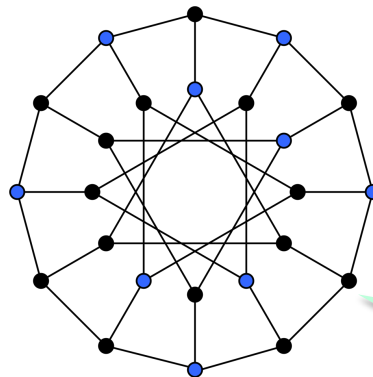
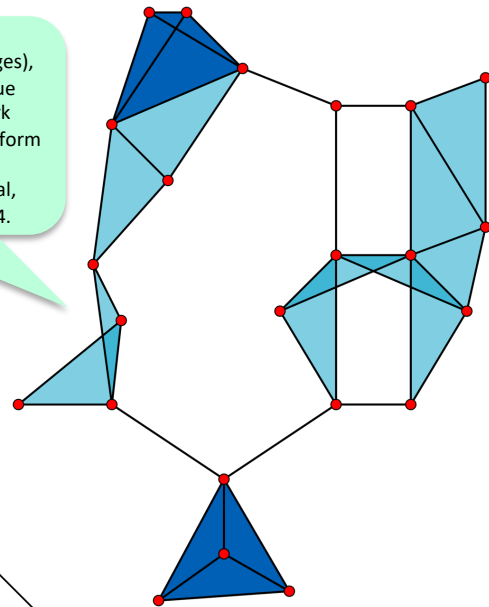
And every clique in G is an independent set in H and vice-versa.

[https://en.wikipedia.org/wiki/Independent_set_\(graph_theory\)](https://en.wikipedia.org/wiki/Independent_set_(graph_theory))

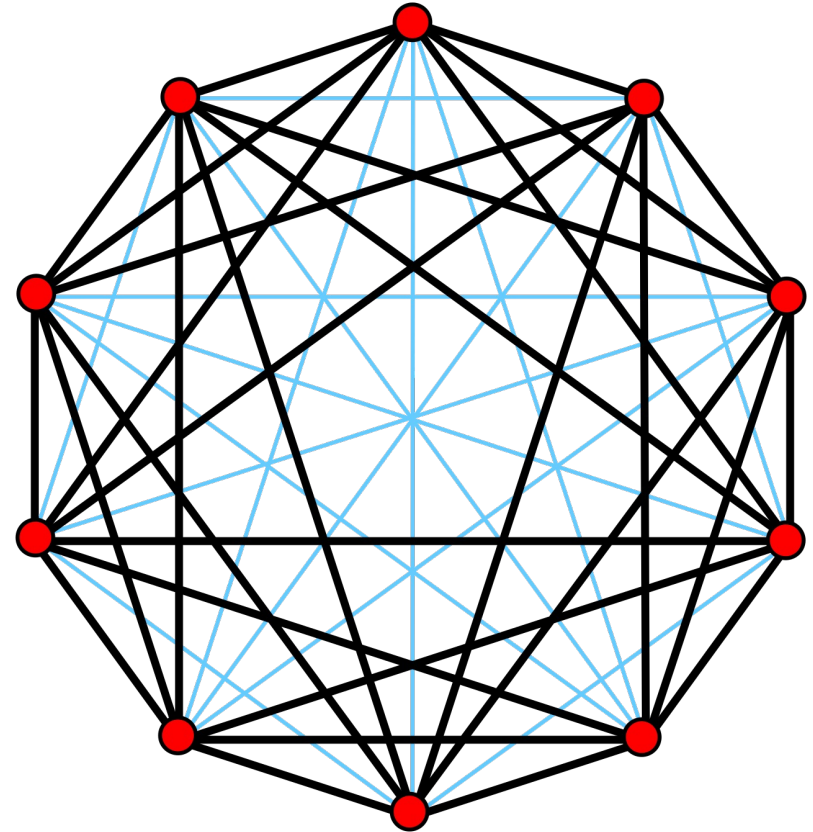
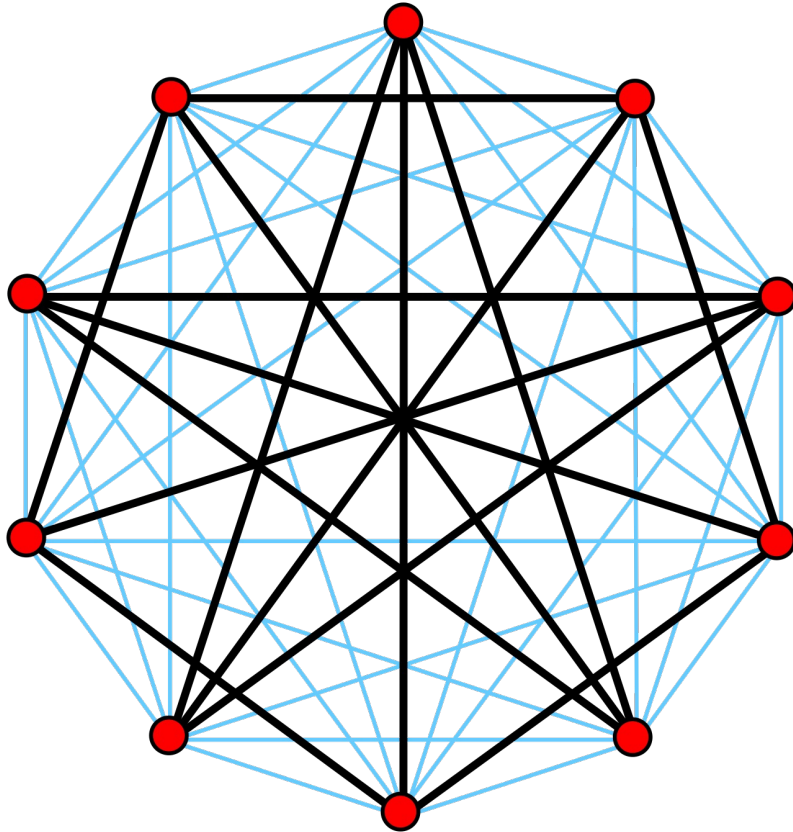
https://en.wikipedia.org/wiki/Complement_graph

<http://archive.dimacs.rutgers.edu/Volumes/Vol26.html>

A graph with 23×1 -vertex cliques (the vertices), 42×2 -vertex cliques (the edges), 19×3 -vertex cliques (light and dark blue triangles), and 2×4 -vertex cliques (dark blue areas). The 11 light blue triangles form maximal cliques. The two dark blue 4-cliques are both maximum and maximal, and the clique number of the graph is 4.



The nine blue vertices form a maximum independent set for the Generalized Petersen graph $GP(12,4)$.



By Complement_graph_sample.gif: Claudio Rocchini derivative work: David Eppstein - Complement_graph_sample.gif, CC BY 2.5, <https://commons.wikimedia.org/w/index.php?curid=8985670>

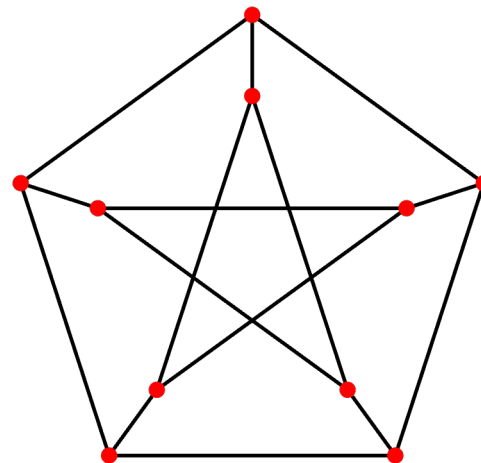
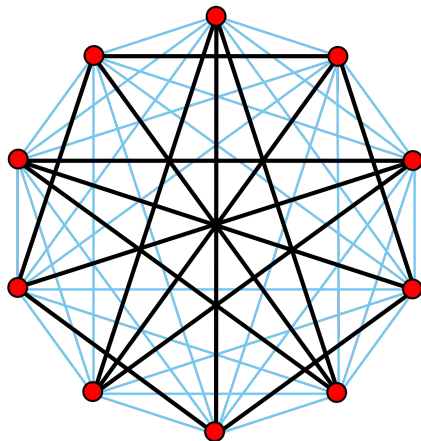
An isomorphism of graphs G and H is a bijection between the vertex sets of G and H

$$f: V(G) \rightarrow V(H)$$

such that any two vertices u and v of G are adjacent in G if and only if $f(u)$ and $f(v)$ are adjacent in H .

The **graph isomorphism problem** is the computational problem of determining whether two finite graphs are isomorphic. The problem is **not known to be solvable in polynomial time nor to be NP-complete**.

Isomorphism for many special classes of graphs can be solved in polynomial time, and in practice graph isomorphism can often be solved efficiently.



Given a graph $G = (V, E)$, find the maximum size independent set of nodes

$$\max_{S \subseteq V} |S| \text{ with } u, v \in S \Rightarrow (u, v) \notin E$$

Max Clique Problem:
We compute the maximum
size independent set on the
complement graph

Binary Linear Programming formulation:

$$\max_{x \in \{0,1\}^{|V|}} \sum_{v \in V} x_v \quad \text{subject to} \quad x_u + x_v \leq 1 \quad \text{for all } (u, v) \in E$$

Unconstraint Quadratic Binary Programming formulation:

$$\min_{x \in \{0,1\}^{|V|}} - \sum_{v \in V} x_v^2 + P \cdot \underbrace{\sum_{(u,v) \in E} x_u \cdot x_v}_{= 0}$$

Graph formulation $G = (V, E, w)$

$$\max_{S, T} \sum_{i \in S, j \in T} w_{ij} \quad \text{with } S \subset V, T \subset V, S \cap T = \emptyset, S \cup T = V$$

Binary Linear Programming formulation:

$$\begin{aligned} \max \quad & \sum_{i=1}^n \sum_{j=1}^n w_{ij} z_{ij} \\ & z_{ij} \leq x_i + x_j \\ & z_{ij} \leq 2 - (x_i + x_j) \\ & x_k \in \{0, 1\} \\ & z_{ij} \in \{0, 1\} \end{aligned}$$

Binary Quadratic Programming formulation:

$$\max_{x \in \{0, 1\}^n} \sum_{i=1}^n \sum_{j=1}^n w_{ij} (x_i + x_j - 2x_i x_j)$$

Can be written as:

$$\min_{x \in \{0, 1\}^n} x^T Q x$$

“The DIMACS Implementation Challenges address questions of determining realistic algorithm performance where worst case analysis is overly pessimistic and probabilistic models are too unrealistic: experimentation can provide guides to realistic algorithm performance where analysis fails.”

The Steiner tree problem in graphs (STP)

Given an undirected connected graph $G = (V, E)$, costs $c: E \rightarrow \mathbb{Q}^+$ and a set $T \subset V$ of *terminals*, find a minimum weight tree $S \subset G$ which spans T .

SCIP-Jack can solve:

- ▷ Steiner Tree Problem in Graphs (STP)
- ▷ Steiner Arborescence Problems in Graphs (SAP)
- ▷ Rectilinear Steiner Minimum Tree (RSMT)
- ▷ Node-weighted Steiner Tree (NWSTP)
- ▷ Prize-collecting Steiner Tree (PCSTP)
- ▷ Rooted Prize-collecting Steiner Tree (RPCSTP)
- ▷ Maximum-weight Connected Subgraph (MWCSP)
- ▷ Degree-constrained Steiner Tree (DCSTP)
- ▷ Group Steiner Tree (GSTP)
- ▷ Hop-constrained directed Steiner Tree (HCDSTP)

$$\begin{aligned}
 & \min \vec{c}^T y \\
 & y(\delta_W^+) \geq 1, & \text{for all } W \subset V, r \in W, (V \setminus W) \cap T \neq \emptyset \\
 & y(\delta_v^-) \begin{cases} = 0, \text{ if } v = r; \\ = 1, \text{ if } v \in T \setminus r; \\ \leq 1, \text{ if } v \in N \end{cases} & \text{for all } v \in V \\
 & y(\delta_v^-) \leq y(\delta_v^+), & \text{for all } v \in N; \\
 & y(\delta_v^-) \geq y_a, & \text{for all } a \in \delta_v^+, v \in N; \\
 & 0 \leq y_a \leq 1, & \text{for all } a \in A; \\
 & y_a \in \{0,1\}, & \text{for all } a \in A,
 \end{aligned}$$

where

$$N = V \setminus T, \delta_X^+ := \{(u, v) \in A \mid u \in X, v \in V \setminus X\}, \delta_X^- := \delta_{V \setminus X}^+ \text{ for } X \subset V.$$

See K. Martin, *Solving Steiner tree problems in graphs to optimality*, 1998, and Polzin, *Algorithms for the Steiner problem in networks*, 2004

RPCSTP

Given an undirected graph $G = (V, E)$, a root $r \in V$, edge-weights $c : E \rightarrow \mathbb{Q}_{\geq 0}$, and node-weights $p : V \rightarrow \mathbb{Q}_{\geq 0}$, a tree $S = (V_S, E_S)$ in G is required such that $r \in S$ and

$$P(S) := \sum_{e \in E_S} c_e + \sum_{v \in V \setminus V_S} p_v$$

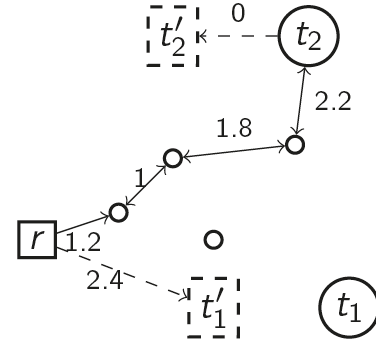
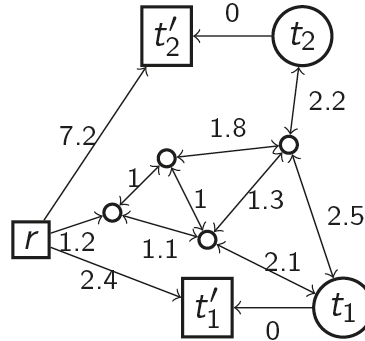
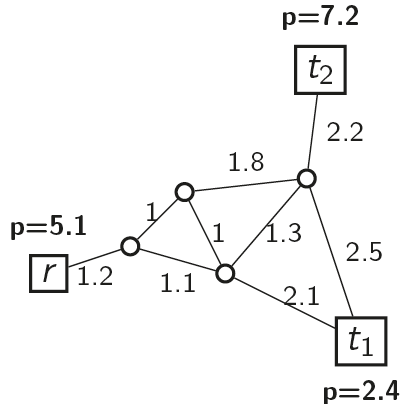
is minimized.

DIMACS fiber network instances, hard instances (>20,000 edges):

- ▷ first publication (Ljubic '04): > 4,000* s.
- ▷ SCIP-Jack at DIMACS (1st) > 100 s
- ▷ best other solver: > 300 s.
- ▷ current SCIP-Jack: < 1 s.

Given RPCSTP $P = (V, E, p, r)$ construct SAP $P' = (V', A', T', c', r')$ as follows:

1. Set $V' := V, A' := \{(v, w) : \{v, w\} \in E\}, r' := r$ and $c' : A' \rightarrow \mathbb{Q}_{\geq 0}$ with $c'_a = c_{\{v, w\}}$ for $a = (v, w) \in A'$.
2. Denote the set of all $v \in V$ with $p_v > 0$ by $T = \{t_1, \dots, t_s\}$.
For each node $t_i \in T$, a new node t'_i and an arc (t_i, t'_i) with $c'_a = 0$ is added to V' and E' respectively.
3. Add arcs (r', t'_i) for each $i \in \{1, \dots, s\}$, setting their respective weight to p_{t_i} .
4. Define the set of terminals $T' := \{t'_1, \dots, t'_s\}$.



```
static X: i32 = 1;
struct X { x: i32 }
impl X {
    fn new() -> X { X { x: X } }
    fn nex(x: i32) -> X { X { x } }
    fn print(&self) {
        print!("{}", self.x)
    }
    fn x(&self) -> i32 { X }
}
fn f(x: i32) -> i32 { x + 1 }
fn g(x: i32) -> i32 { X + x }

fn main() {
    print!("{}", X);
    let mut x = 2;
    print!("{}", x);
    {
```

```
        for x in 1..7 {
        }
        print!("{}", x);
        let x = 3;
        print!("{}", x);
    }
    let x = X::nex(3);
    x.print();
    print!("{}", g(x.x() + X));
    print!("{}", f(x.x()));
    {
        print!("{}", g(X));
        let x = X::new();
        x.print();
    }
    println!();
}
```

122333221

```
static X: i32 = 1;
struct X { x: i32 }
impl X {
    fn new() -> X { X { x: 1 } }
    fn nex(x: i32) -> X { X { x } }
    fn print(&self) {
        print!("{}", self.x)
    }
    fn x(&self) -> i32 { 1 }
}
fn f(x: i32) -> i32 { x + 1 }
fn g(x: i32) -> i32 { 1 + x }

fn main() {
    print!("{}", 1);
    let mut x = 2;
    print!("{}", x);
    {
```

```
        for x in 1..7 {
            }
            print!("{}", x);
            let x = 3;
            print!("{}", x);
        }
        let x = X::nex(3);
        x.print();
        print!("{}", g(x.x() + 1));
        print!("{}", f(x.x()));
        {
            print!("{}", g(1));
            let x = X::new();
            x.print();
        }
        println!();
    }
```

```
struct X { x: i32 }
impl X {
    fn new() -> X { X { x: 1 } }
    fn nex(x: i32) -> X { X { x } }
    fn print(&self) {
        print!("{}", self.x)
    }
    fn x(&self) -> i32 { 1 }
}
fn f(x: i32) -> i32 { x + 1 }
fn g(x: i32) -> i32 { 1 + x }

fn main() {
    print!("{}", 1); // -> 1
    let mut x = 2;
    print!("{}", x); // -> 2
    {
```

```
        print!("{}", x); // -> 2
        let x = 3;
        print!("{}", x); // -> 3
    }
    let x = X::nex(3);
    x.print(); // -> 3
    print!("{}", f(1 + 1)); // -> 3
    print!("{}", f(1)); // -> 2
    {
        print!("{}", f(1)); // -> 2
        let x = X::new();
        x.print(); // -> 1
    }
    println!();
}
```

Student Presentation


```
let mut i = num_items;
```

```
while i > 0 {  
    if dp[i][cap] != dp[i - 1][cap] {  
        taken.push(i);  
        cap -= w[i - 1];  
        i -= 1  
    } else {  
        i -= 1;  
    }  
}
```

```
let mut i = num_items;
```

```
while i > 0 {  
    if dp[i][cap] != dp[i - 1][cap] {  
        taken.push(i);  
        cap -= w[i - 1];  
    }  
    i -= 1;  
}
```

```
for i in (1 ..= num_items).rev() {  
    if dp[i][cap] != dp[i - 1][cap] {  
        taken.push(i);  
        cap -= w[i - 1];  
    }  
}
```

```
for j in 0 ..= capacity {  
  if i == 0 || j == 0 {  
    knap[i][j] = 0;  
    continue;  
  }  
  if i == 0 {  
    continue;  
  } else if weight[i] > j {  
    knap[i][j] = knap[i-1][j];  
    continue;  
  } else {  
    let tmp1 = knap[i-1][j];  
    let tmp2 = knap[i-1][j - weight[i]] + val[i];  
  
    if tmp1 > tmp2 {  
      knap[i][j] = tmp1;  
      continue;  
    } else {  
      knap[i][j] = tmp2;  
      continue;  
    }  
  }  
}
```

```
for j in 0 ..= capacity {  
  if i == 0 || j == 0 {  
    knap[i][j] = 0;  
    continue;  
  }  
  if weight[i] > j {  
    knap[i][j] = knap[i-1][j];  
    continue;  
  }  
  let tmp1 = knap[i-1][j];  
  let tmp2 = knap[i-1][j - weight[i]] + val[i];  
  
  if tmp1 > tmp2 {  
    knap[i][j] = tmp1;  
    continue;  
  }  
  knap[i][j] = tmp2;  
}
```

```
for j in 0 ..= capacity {  
  if i == 0 || j == 0 {  
    knap[i][j] = 0;  
    continue;  
  }  
  if i == 0 {  
    continue;  
  } else if weight[i] > j {  
    knap[i][j] = knap[i-1][j];  
    continue;  
  } else {  
    let tmp1 = knap[i-1][j];  
    let tmp2 = knap[i-1][j - weight[i]] + val[i];  
  
    if tmp1 > tmp2 {  
      knap[i][j] = tmp1;  
      continue;  
    } else {  
      knap[i][j] = tmp2;  
      continue;  
    }  
  }  
}
```

```
for j in 0 ..= capacity {  
  if i == 0 || j == 0 {  
    knap[i][j] = 0;  
  } else if weight[i] > j {  
    knap[i][j] = knap[i-1][j];  
  } else {  
    let tmp1 = knap[i-1][j];  
    let tmp2 = knapsack[i-1][j - weight[i]] + val[i];  
  
    if tmp1 > tmp2 {  
      knap[i][j] = tmp1;  
    } else {  
      knap[i][j] = tmp2;  
    }  
  }  
}
```

```
for j in 0 ..= capacity {
  if i == 0 || j == 0 {
    knap[i][j] = 0;
    continue;
  }
  if i == 0 {
    continue;
  } else if weight[i] > j {
    knap[i][j] = knap[i-1][j];
    continue;
  } else {
    let tmp1 = knap[i-1][j];
    let tmp2 = knap[i-1][j - weight[i]] + val[i];

    if tmp1 > tmp2 {
      knap[i][j] = tmp1;
      continue;
    } else {
      knap[i][j] = tmp2;
      continue;
    }
  }
}
```

```
for j in 0 ..= capacity {
  if i == 0 || j == 0 {
    knap[i][j] = 0;
    continue;
  }
  if weight[i] > j {
    knap[i][j] = knap[i-1][j];
    continue;
  }
  let tmp1 = knap[i-1][j];
  let tmp2 = knap[i-1][j - weight[i]] + val[i];

  knap[i][j] = if tmp1 > tmp2 { tmp1 } else { tmp2 };
}
```

```
for ... {  
    if arr[i][w] == arr[i - 1][w] {  
        continue;  
    } else {  
        itemlist.push_back(i);  
        w -= vals[i - 1][2];  
    }  
}
```

```
for ... {  
    if arr[i][w] == arr[i - 1][w] {  
        continue;  
    }  
    itemlist.push_back(i);  
    w -= vals[i-1][2];  
}
```

```
for ... {  
    if arr[i][w] != arr[i - 1][w] {  
        itemlist.push_back(i);  
        w -= vals[i-1][2];  
    }  
}
```

Fragen

有問題嗎

คำถาม

?

質問

Вопросы

Questions

Câu hỏi